

AfroStore Gap Analysis Report

Generated: July 22, 2026

Version: 1.0

Analysis Type: Pre-Launch Implementation Review

Executive Summary

This report analyzes the current implementation of AfroStore against the PRD Phase 1 MVP requirements. The analysis reveals significant progress in core functionality but identifies critical gaps in security, infrastructure, and operational readiness that must be addressed before launch.

Overall Status:

- Core Ecommerce Features: 85% Complete
 - Security Infrastructure: 40% Complete
 - Production Readiness: 60% Complete
 - Documentation: 20% Complete
-

Critical Security Gaps

Authentication & Security

Email Verification Flow

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - User onboarding security vulnerability
- **Evidence:** No email verification implementation found in codebase
- **Recommendation:** Implement email verification using email service provider

Password Reset Flow

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Critical user experience and security feature
- **Evidence:** No password reset functionality implemented
- **Recommendation:** Implement password reset with email service integration

CSRF Protection

- **Status:** NOT IMPLEMENTED
- **Impact:** CRITICAL - Security vulnerability
- **Evidence:** No CSRF protection found in codebase
- **Recommendation:** Implement CSRF tokens for all state-changing operations

Rate Limiting

- **Status:** NOT IMPLEMENTED
- **Impact:** CRITICAL - API security and abuse prevention

- **Evidence:** No API rate limiting implementation (only found in AI failover)
- **Recommendation:** Implement rate limiting using Redis or similar

XSS Protection

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Security vulnerability
- **Evidence:** No explicit XSS protection measures
- **Recommendation:** Implement input sanitization and output encoding

SQL Injection Protection

- **Status:** △ PARTIAL
 - **Impact:** MEDIUM - Relies solely on Prisma ORM
 - **Evidence:** Only Prisma ORM protection, no additional validation layers
 - **Recommendation:** Add additional input validation and query parameter sanitization
-

Infrastructure & Operations

Caching & Performance

Redis Integration

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Performance and scalability limitation
- **Evidence:** No Redis for caching/sessions found
- **Recommendation:** Implement Redis for session management, caching, and rate limiting

CDN Configuration

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Performance and global reach limitation
- **Evidence:** No CDN integration implemented
- **Recommendation:** Integrate CDN (Cloudflare, AWS CloudFront) for static assets

Image Optimization Service

- **Status:** △ MINIMAL
- **Impact:** MEDIUM - Performance and bandwidth costs
- **Evidence:** Only basic middleware, no comprehensive optimization
- **Recommendation:** Implement dedicated image optimization service (Cloudinary, imgix)

Caching Strategy

- **Status:** △ BASIC
- **Impact:** MEDIUM - Performance limitation
- **Evidence:** No systematic caching beyond Next.js defaults
- **Recommendation:** Implement comprehensive caching strategy (Redis, CDN, HTTP caching)

Background Jobs & Queues

Queue System

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Scalability and reliability limitation
- **Evidence:** No background job queue system (Bull, Agenda, etc.)
- **Recommendation:** Implement queue system for async tasks (emails, reports, sync)

Background Job Processing

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Performance and user experience limitation
- **Evidence:** No async job infrastructure
- **Recommendation:** Implement background job processing infrastructure

Monitoring & Error Tracking

Error Tracking (Sentry)

- **Status:** NOT IMPLEMENTED
- **Impact:** CRITICAL - Production debugging and stability
- **Evidence:** No error tracking service integration
- **Recommendation:** Implement Sentry or similar error tracking service

Comprehensive Monitoring

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Operational visibility
- **Evidence:** No monitoring setup (Datadog, New Relic, etc.)
- **Recommendation:** Implement APM and infrastructure monitoring

Performance Monitoring

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Performance optimization
- **Evidence:** No APM or performance tracking
- **Recommendation:** Implement performance monitoring and alerting

Log Aggregation

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Debugging and troubleshooting
- **Evidence:** No centralized logging system
- **Recommendation:** Implement log aggregation (ELK stack, CloudWatch Logs)

Communication Services

Email Service

Email Service Integration

- **Status:** NOT IMPLEMENTED

- **Impact:** CRITICAL - User communication and notifications
- **Evidence:** No Resend, SendGrid, Mailgun, or similar service
- **Recommendation:** Implement email service provider (Resend recommended)

Transactional Emails

- **Status:** NOT IMPLEMENTED
- **Impact:** CRITICAL - Order confirmations, notifications
- **Evidence:** No email sending capability
- **Recommendation:** Implement transactional email system

Email Templates

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Brand consistency and user experience
- **Evidence:** No email template system
- **Recommendation:** Create email template system for all communications

Notification Emails

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - User engagement and support
- **Evidence:** No email notifications for orders, password reset, etc.
- **Recommendation:** Implement notification email system

Testing & Quality Assurance

Test Coverage

E2E Test Execution

- **Status:** △ FRAMEWORK EXISTS
- **Impact:** MEDIUM - Quality assurance
- **Evidence:** Test framework exists but actual test coverage unknown
- **Recommendation:** Expand E2E test coverage to critical paths

Unit Test Coverage

- **Status:** △ MINIMAL
- **Impact:** MEDIUM - Code quality and regression prevention
- **Evidence:** Minimal unit tests found
- **Recommendation:** Increase unit test coverage to 70%+

Integration Tests

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - System integration validation
- **Evidence:** No integration test suite
- **Recommendation:** Implement integration tests for API endpoints

Performance Testing

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Performance validation
- **Evidence:** No load testing or performance benchmarks
- **Recommendation:** Implement load testing and performance benchmarks

Security Testing

- **Status:** NOT IMPLEMENTED
 - **Impact:** HIGH - Security validation
 - **Evidence:** No security audit or penetration testing
 - **Recommendation:** Conduct security audit and penetration testing
-

Deployment & DevOps

Deployment Infrastructure

Automated Backup Strategy

- **Status:** NOT IMPLEMENTED
- **Impact:** CRITICAL - Data safety and disaster recovery
- **Evidence:** No automated database backup system
- **Recommendation:** Implement automated daily backups with retention policy

Database Migration Automation

- **Status:** △ MANUAL
- **Impact:** MEDIUM - Deployment reliability
- **Evidence:** Manual migrations only
- **Recommendation:** Automate database migrations in deployment pipeline

CI/CD Pipeline

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Deployment efficiency and reliability
- **Evidence:** No automated deployment pipeline
- **Recommendation:** Implement CI/CD pipeline (GitHub Actions, GitLab CI)

Environment-Specific Configs

- **Status:** △ BASIC
- **Impact:** MEDIUM - Configuration management
- **Evidence:** Basic env vars but no config management system
- **Recommendation:** Implement configuration management system

SSL Certificate Automation

- **Status:** △ UNCLEAR
- **Impact:** MEDIUM - SSL management
- **Evidence:** SSL manager exists but automation unclear
- **Recommendation:** Implement Let's Encrypt automation

Production Readiness

Database Connection Pooling

- **Status:** ⚠️ DEFAULTS
- **Impact:** MEDIUM - Database performance
- **Evidence:** Relies on Prisma defaults
- **Recommendation:** Optimize database connection pooling for production load

Health Check Endpoints

- **Status:** ⚠️ LIMITED
- **Impact:** MEDIUM - System monitoring
- **Evidence:** Limited health check implementation
- **Recommendation:** Implement comprehensive health check endpoints

Graceful Shutdown

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Deployment reliability
- **Evidence:** No graceful shutdown handling
- **Recommendation:** Implement graceful shutdown for zero-downtime deployments

Disaster Recovery Plan

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Business continuity
- **Evidence:** No DR procedures documented
- **Recommendation:** Document and test disaster recovery procedures

Documentation

User Documentation

Getting Started Guide

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - User onboarding
- **Evidence:** Only 2 technical docs found
- **Recommendation:** Create comprehensive getting started guide

Site Creation Tutorial

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - User onboarding
- **Evidence:** No user-facing documentation
- **Recommendation:** Create step-by-step site creation tutorial

Payment Setup Guide

- **Status:** NOT IMPLEMENTED

- **Impact:** HIGH - User enablement
- **Evidence:** No payment integration guides
- **Recommendation:** Create payment gateway setup guides

Theme Customization Guide

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - User empowerment
- **Evidence:** No theme documentation
- **Recommendation:** Create theme customization documentation

FAQ

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Support efficiency
- **Evidence:** No FAQ documentation
- **Recommendation:** Create comprehensive FAQ

Developer Documentation

API Documentation

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Developer ecosystem
- **Evidence:** No comprehensive API docs
- **Recommendation:** Create API documentation (Swagger/OpenAPI)

Plugin Development Guide

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Ecosystem growth
- **Evidence:** No plugin dev documentation
- **Recommendation:** Create plugin development guide

Theme Development Guide

- **Status:** NOT IMPLEMENTED
- **Impact:** MEDIUM - Ecosystem growth
- **Evidence:** No theme dev documentation
- **Recommendation:** Create theme development guide

Deployment Guide

- **Status:** NOT IMPLEMENTED
- **Impact:** HIGH - Self-hosting capability
- **Evidence:** No deployment documentation
- **Recommendation:** Create deployment and infrastructure guide

Architecture Documentation

- **Status:** △ LIMITED
- **Impact:** MEDIUM - System understanding

- **Evidence:** Limited system documentation
 - **Recommendation:** Create comprehensive architecture documentation
-

Partially Implemented

Analytics

- **Status:** ⚠ PARTIAL
- **Implementation:**
 - Analytics dashboard exists
 - Event tracking implemented
 - ⚠ Conversion tracking partially implemented
 - ⚠ Comprehensive tracking unclear
- **Gap:** Need comprehensive analytics implementation and success metrics tracking

Webhooks

- **Status:** IMPLEMENTED
- **Implementation:**
 - Paystack webhooks
 - Monnify webhooks
 - Flutterwave webhooks
 - Signature verification
- **Gap:** None - well implemented

SSL/Domains

- **Status:** ⚠ PARTIAL
 - **Implementation:**
 - SSL manager exists
 - Domain verification endpoints
 - ⚠ Automation unclear
 - **Gap:** Need SSL certificate automation
-

Implemented & Verified

Core Features (Working)

- User authentication (signup/login)
- Multi-site architecture
- Template system with sample data
- Product management
- Order management
- Payment gateway integrations (Paystack, Flutterwave, Monnify)
- AI service with failover

- Drag-and-drop builder
 - Africa-specific features (WhatsApp, delivery zones)
 - Admin platform
 - Plugin system architecture
 - Theme system
 - Basic analytics
 - E2E test framework (setup exists)
-

Immediate Blockers for Launch

Critical (Must Fix Before Launch)

1. **Email service** - Critical for password reset, order confirmations, notifications
2. **CSRF protection** - Critical security vulnerability
3. **Rate limiting** - Critical for API security and abuse prevention
4. **Error tracking** - Critical for production debugging
5. **Redis/caching** - Important for performance at scale
6. **Email verification** - Important for user onboarding security
7. **Backup strategy** - Critical for data safety
8. **Comprehensive testing** - Critical for stability

High Priority (Launch + 30 Days)

1. Implement Redis for caching
2. Add comprehensive monitoring
3. Set up CDN integration
4. Create user documentation
5. Implement background job queue
6. Add image optimization service
7. Set up CI/CD pipeline

Medium Priority (Post-Launch)

1. Expand unit test coverage
 2. Implement performance monitoring
 3. Create developer documentation
 4. Add security audit processes
 5. Implement disaster recovery procedures
-

Recommended Launch Priority

Phase 0 (Must Have Before Launch) - 2-3 Weeks

Estimated Effort: 80-120 hours

Security:

- Implement email service integration (16h)
- Add CSRF protection (8h)
- Implement API rate limiting (12h)

- Implement email verification flow (8h)
- Add password reset flow (8h)

Infrastructure:

- Set up error tracking (Sentry) (8h)
- Set up automated database backups (8h)
- Implement Redis for basic caching (12h)

Testing:

- Expand E2E test coverage (20h)
- Add security audit (8h)

Documentation:

- Create getting started guide (8h)
- Create payment setup guides (8h)

Phase 1 (Launch + 30 Days) - 4-6 Weeks

Estimated Effort: 120-160 hours

Performance:

- Implement comprehensive caching strategy (16h)
- Set up CDN integration (12h)
- Add image optimization service (16h)
- Implement performance monitoring (12h)

Operations:

- Implement background job queue (16h)
- Set up CI/CD pipeline (16h)
- Add comprehensive monitoring (16h)
- Implement log aggregation (12h)

Documentation:

- Create user documentation suite (24h)
- Create API documentation (16h)

Phase 2 (Post-Launch) - Ongoing

Estimated Effort: 40-60 hours/month

Quality:

- Expand unit test coverage (ongoing)
- Implement performance testing (8h)
- Add security audit processes (quarterly)

Documentation:

- Create developer documentation (16h)
- Create architecture documentation (12h)

Operations:

- Implement disaster recovery procedures (16h)
- Optimize infrastructure (ongoing)

Gap Summary

By Category

Category	Status	Completion	Priority
Security	Critical	40%	Phase 0
Infrastructure	Critical	50%	Phase 0
Communication	Critical	20%	Phase 0
Testing	High	30%	Phase 0
Deployment	High	60%	Phase 1
Documentation	Critical	20%	Phase 1
Core Features	Complete	85%	N/A
AI Features	Complete	80%	N/A

By Impact

Impact Level	Count	Items
Critical	8	Email service, CSRF, Rate limiting, Error tracking, Backups, Email verification, Password reset, XSS protection
High	12	Redis, CDN, Image optimization, Monitoring, Testing coverage, Documentation, SSL automation, Health checks, etc.
Medium	15	Performance monitoring, Log aggregation, CI/CD, Graceful shutdown, Unit tests, etc.

Success Criteria

Before Launch

- ☐ All critical security gaps addressed
- ☐ Email service fully functional
- ☐ Comprehensive test coverage (70%+)
- ☐ Automated backup system operational
- ☐ Error tracking implemented
- ☐ Basic documentation complete

Launch + 30 Days

- ☐ Redis caching implemented
- ☐ CDN integration complete
- ☐ Monitoring and alerting operational
- ☐ Background job queue functional
- ☐ User documentation complete
- ☐ CI/CD pipeline operational

Launch + 90 Days

- [] Performance optimization complete
 - [] Developer documentation complete
 - [] Disaster recovery tested
 - [] Security audit passed
 - [] Load testing complete
-

Recommendations

Immediate Actions

1. **Prioritize security gaps** - Address CSRF, rate limiting, and email service immediately
2. **Implement error tracking** - Essential for production stability
3. **Set up backups** - Critical for data safety before launch
4. **Expand testing** - Increase test coverage for critical paths

Strategic Actions

1. **Invest in infrastructure** - Redis, CDN, monitoring are foundational
2. **Build documentation** - Essential for user onboarding and ecosystem growth
3. **Automate operations** - CI/CD, backups, monitoring reduce operational burden
4. **Plan for scale** - Queue system, caching, monitoring prepare for growth

Risk Mitigation

1. **Security audit** - Conduct professional security audit before launch
 2. **Load testing** - Validate performance under expected load
 3. **Disaster recovery** - Test backup and restore procedures
 4. **Support plan** - Establish support processes and escalation paths
-

Conclusion

The AfroStore implementation demonstrates strong progress in core ecommerce functionality and innovative AI features. However, critical gaps in security infrastructure, communication services, and operational readiness must be addressed before launch.

Recommended Timeline: 6-8 weeks to address critical gaps for a production-ready launch.

Key Focus: Security first, then infrastructure, then documentation and testing.

Risk Level: HIGH without addressing security and infrastructure gaps.

Overall Assessment: Foundation is solid, but production readiness requires significant investment in security, infrastructure, and operational maturity.

This gap analysis is based on codebase review against PRD Phase 1 MVP requirements